

Testing Plan for the Removals System

Mindaugas Taujanskas

The chosen testing framework is PYTEST. No performance testing, code coverage, or comprehensive end-to-end testing will be considered. A virtual environment must be used to ensure replicability of the precise dependencies and configuration. A MAKEFILE must supplement this virtual environment with the necessary rules to configure the software/database environment.

I. DATABASE INTEGRITY

A. FT001: *Connection*

- 1) TC01: Connectivity
Ensure database connectivity is explicitly error-handled, and propagated to the appropriate layer which must be visible to the user.
- 2) TC02: Initialised schema
Ensure database schema is appropriately initialised to enable further program functionality; prevent subsequent execution of the software if not.

II. USER AUTHENTICATION

A. FT101: *Login*

- 1) TC01: Invalid credentials
Ensure the user cannot proceed to the dashboard with invalid credentials, displaying a clear error-state on failure.
- 2) TC02: Service-provider pending approval
Ensure a service provider who is not approved cannot proceed to the dashboard, displaying a clear indication as to why.
- 3) TC03: Deactivated user
Ensure any user who is deactivated (whether deleted/banned) cannot proceed to the dashboard, displaying a clear indication as to why.

B. FT102: *Registration*

- 1) TC01: Invalid fields¹
Ensure required fields which are empty, misformatted fields, or otherwise fields violating their specific validation constraints are clearly identified, and propagated to the user clearly, preventing the registration process.
- 2) TC02: Existing email
Ensure that duplicate registration with the same email (wrt. its canonical form²) is disallowed, and propagated to the user clearly, preventing the registration process.

¹ TC01 applies broadly, as registration branches depending on whether the user selects customer or service-provider registration, and the subsequent fields have a plethora of trivial validation constraints dictated by the database schema.

² TC02 refers to the canonical form of an email, meaning that an email such as `alice+1@example.com` will be reduced to `alice@example.com`, as they refer to the same source email address, and thus must not be allowed for registration.

III. DATABASE ACCESS CONTROLS

A. FT201: *Raw table access*

- 1) TC01: No DDL/DML operations
Ensure the top-level application layer cannot execute DDL/DML SQL statements against the database.
- 2) TC02: Access to exposed SQL procedures
Ensure the top-level application layer can execute exposed SQL procedures.

B. FT202: *Schema access*

- 1) TC02: Restricted to public schema
Ensure the user cannot execute SQL procedures outside of the publicly exposed schema.

IV. USER MODELS

Note that for each user model, there will tend to be sub-classifications which are hard-set by the database, e.g. phone numbers must be identified as for either home/work, therefore these cases must also be accounted for.

A. FT301: *Phone numbers*

- 1) TC01: Creation
Ensure that phone numbers can be created and attributed to a valid registered user.
- 2) TC02: Retrieval
Ensure that a user's phone numbers can be retrieved accurately.

B. FT302: *Addresses*

- 1) TC01: Creation
Ensure that addresses can be created and attributed to a valid registered user.
- 2) TC02: Retrieval
Ensure that a user's addresses can be retrieved accurately.

V. USER ACCEPTANCE

A simple user-flow should be demonstrable of both customer registration and subsequent sign-in to display the user dashboard. Successful service-provider registration should display a pending approval view, and sign-in should not succeed.